EXP8:

```
import numpy as np

import pandas as pd

import matplotlib.pyplot as plt

from sklearn import datasets

from sklearn.cluster import KMeans


# ----------------- LOAD DATASET -----------------

iris = datasets.load_iris()

x = pd.DataFrame(iris.data)

x.columns = ['sepal_length', 'sepal_width', 'petal_length', 'petal_width']


plt.figure(figsize=(15, 10))


# ----------------- TEST CASE 1: TWO CLUSTERS -----------------

model1 = KMeans(n_clusters=2, random_state=0)

model1.fit(x)


plt.subplot(2, 3, 1)

plt.scatter(x['petal_length'], x['petal_width'], c=model1.labels_, cmap='viridis')

plt.scatter(model1.cluster_centers_[:, 2], model1.cluster_centers_[:, 3], color='red', marker='x',
s=100)

plt.title("TC1: 2 Centroids")


# ----------------- TEST CASE 2: THREE CLUSTERS (ORIGINAL) -----------------

model2 = KMeans(n_clusters=3, random_state=0)

model2.fit(x)


plt.subplot(2, 3, 2)
```

```
plt.scatter(x['petal_length'], x['petal_width'], c=model2.labels_, cmap='viridis')
plt.scatter(model2.cluster_centers_[:, 2], model2.cluster_centers_[:, 3], color='red', marker='x', s=100)
plt.title("TC2: 3 Centroids")

# ---------- TEST CASE 3: CENTROID CONVERGENCE ----------
model3 = KMeans(n_clusters=3, max_iter=300, random_state=0)
model3.fit(x)

plt.subplot(2, 3, 3)
plt.scatter(x['petal_length'], x['petal_width'], c=model3.labels_, cmap='viridis')
plt.scatter(model3.cluster_centers_[:, 2], model3.cluster_centers_[:, 3], color='red', marker='x', s=100)
plt.title("TC3: Converged")

print("\nTC3: Convergence Test")
print("Number of iterations: ", model3.n_iter_)
print("Centroids: ")
print(model3.cluster_centers_)

# ---------- TEST CASE 4: K > DATA POINTS ----------
plt.subplot(2, 3, 4)
try:
    x_small = x.head(5)
    model4 = KMeans(n_clusters=10, random_state=0)
    model4.fit(x_small)
except ValueError:
    plt.text(0.2, 0.5, "Error: k > data points", fontsize=10)
    plt.title("TC4: Error")
```

```
# ----------------- TEST CASE 5: DUPLICATE POINTS -----------------

x_dup = pd.concat([x, x.iloc[:50], x.iloc[:50], x.iloc[:50]])

print("\nOriginal Records: ", len(x))

print("Records After Adding Duplicates: ", len(x_dup))


model5 = KMeans(n_clusters=3, random_state=0)

model5.fit(x_dup)


plt.subplot(2, 3, 5)

plt.scatter(x_dup['petal_length'], x_dup['petal_width'], c=model5.labels_, cmap='viridis')

plt.scatter(model5.cluster_centers_[:, 2], model5.cluster_centers_[:, 3], color='red', marker='x',
s=100)

plt.title("TC5: Many Duplicate Points")


plt.tight_layout()

plt.show()
```

EXP:9

```
import numpy as np

import pandas as pd

import matplotlib.pyplot as plt

from sklearn import datasets

from sklearn.cluster import KMeans


# ----------------- LOAD DATASET -----------------

iris = datasets.load_iris()

x = pd.DataFrame(iris.data)

x.columns = ['sepal_length', 'sepal_width', 'petal_length', 'petal_width']


plt.figure(figsize=(15, 10))


# ----------------- TEST CASE 1: TWO CLUSTERS -----------------

model1 = KMeans(n_clusters=2, random_state=0)

model1.fit(x)


plt.subplot(2, 3, 1)

plt.scatter(x['petal_length'], x['petal_width'], c=model1.labels_, cmap='viridis')

plt.scatter(model1.cluster_centers_[:, 2], model1.cluster_centers_[:, 3], color='red', marker='x',
s=100)

plt.title("TC1: 2 Centroids")


# ----------------- TEST CASE 2: THREE CLUSTERS (ORIGINAL) -----------------

model2 = KMeans(n_clusters=3, random_state=0)

model2.fit(x)


plt.subplot(2, 3, 2)
```

```
plt.scatter(x['petal_length'], x['petal_width'], c=model2.labels_, cmap='viridis')
plt.scatter(model2.cluster_centers_[:, 2], model2.cluster_centers_[:, 3], color='red', marker='x', s=100)
plt.title("TC2: 3 Centroids")

# ---------- TEST CASE 3: CENTROID CONVERGENCE ----------
model3 = KMeans(n_clusters=3, max_iter=300, random_state=0)
model3.fit(x)

plt.subplot(2, 3, 3)
plt.scatter(x['petal_length'], x['petal_width'], c=model3.labels_, cmap='viridis')
plt.scatter(model3.cluster_centers_[:, 2], model3.cluster_centers_[:, 3], color='red', marker='x', s=100)
plt.title("TC3: Converged")

print("\nTC3: Convergence Test")
print("Number of iterations: ", model3.n_iter_)
print("Centroids: ")
print(model3.cluster_centers_)

# ---------- TEST CASE 4: K > DATA POINTS ----------
plt.subplot(2, 3, 4)
try:
    x_small = x.head(5)
    model4 = KMeans(n_clusters=10, random_state=0)
    model4.fit(x_small)
except ValueError:
    plt.text(0.2, 0.5, "Error: k > data points", fontsize=10)
    plt.title("TC4: Error")
```

```
# ----------------- TEST CASE 5: DUPLICATE POINTS -----------------

x_dup = pd.concat([x, x.iloc[:50], x.iloc[:50], x.iloc[:50]])

print("\nOriginal Records: ", len(x))

print("Records After Adding Duplicates: ", len(x_dup))


model5 = KMeans(n_clusters=3, random_state=0)

model5.fit(x_dup)


plt.subplot(2, 3, 5)

plt.scatter(x_dup['petal_length'], x_dup['petal_width'], c=model5.labels_, cmap='viridis')

plt.scatter(model5.cluster_centers_[:, 2], model5.cluster_centers_[:, 3], color='red', marker='x',
s=100)

plt.title("TC5: Many Duplicate Points")


plt.tight_layout()

plt.show()
```

EXP12:

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score

# ----------------- LOAD DATASET -----------------
iris = load_iris()
x, y = iris.data, iris.target

# ============================================================
# TC1: Linearly Separable Data (Classes 0 and 1 only)
# ============================================================
x1, y1 = x[y != 2], y[y != 2]

model1 = SVC(kernel='linear')
model1.fit(x1, y1)
print("TC1 Accuracy :", accuracy_score(y1, model1.predict(x1)))

# ============================================================
# TC2: Two-Class Dataset (Train-Test Split)
# ============================================================
x_train, x_test, y_train, y_test = train_test_split(
    x1, y1, test_size=0.2, random_state=42
)

model2 = SVC(kernel='linear')
```

```
model2.fit(x_train, y_train)
print("TC2 Accuracy :", accuracy_score(y_test, model2.predict(x_test)))

# ========================================================
# TC3: Test Point Near Margin
# ========================================================
model3 = SVC(kernel='linear')
model3.fit(x1, y1)

# Point from the dataset located near the decision boundary
point = [x1[23]]

print("TC3 predicted class: ", model3.predict(point)[0])
print("TC3 decision value : ", model3.decision_function(point)[0])

# ========================================================
# TC4: Overlapping Classes (Added Noise)
# ========================================================
np.random.seed(42)
x4 = x1 + np.random.normal(0, 0.8, x1.shape)

model4 = SVC(kernel='linear')
model4.fit(x4, y1)
print("TC4 Accuracy :", accuracy_score(y1, model4.predict(x4)))

# ========================================================
# TC5: Single-Class Data
# ========================================================
x5, y5 = x[y == 0], y[y == 0]
```

```
try:
   model5 = SVC(kernel='linear')
   model5.fit(x5, y5)
except ValueError:
   print("TC5: SVM needs at least two classes.")

# ============================================================
# VISUALIZATION (Decision Boundary & Margins)
# ============================================================
# Use first 2 features (Sepal length & Sepal width) for 2D plotting
xg = x1[:, :2]
graph_model = SVC(kernel='linear')
graph_model.fit(xg, y1)

# Scatter plot data points
plt.scatter(xg[y1 == 0, 0], xg[y1 == 0, 1], label="Class 0")
plt.scatter(xg[y1 == 1, 0], xg[y1 == 1, 1], label="Class 1")

# Compute decision hyperplane and margin
xx = np.linspace(xg[:, 0].min(), xg[:, 0].max(), 100)
w = graph_model.coef_[0]
b = graph_model.intercept_[0]

yy = -(w[0] * xx + b) / w[1]
plt.plot(xx, yy, linewidth=2, label="Hyperplane")

yy_lower = -(w[0] * xx + b + 1) / w[1]
plt.plot(xx, yy_lower, '--', label="margin - 1")

# Highlight support vectors
```

```
plt.scatter(
    graph_model.support_vectors_[:, 0],
    graph_model.support_vectors_[:, 1],
    facecolors='none',
    edgecolors='black',
    s=100
)

# Graph Labels
plt.xlabel("Sepal length")
plt.ylabel("Sepal width")
plt.title("SVM Classification")
plt.legend()
plt.grid(True)
plt.show()
```

EXP:5

```
# Simple Bayesian Network
# Burglary and Earthquake probabilities
P_B = 0.002
P_E = 0.001

# Alarm Conditional Probability Table (CPT)
alarm = {
   (True, True): 0.95,
   (True, False): 0.94,
   (False, True): 0.29,
   (False, False): 0.001
}

# David and Sophia call probabilities
P_David = 0.90
P_Sophia = 0.70

# ----------------- TEST CASE 1 -----------------
print("Test case 1")
B = False
E = False
A = True
D = True
S = True

p = (1 - P_B) * (1 - P_E) * alarm[(B, E)] * P_David * P_Sophia
print("Probability = ", p)

# ----------------- TEST CASE 2 -----------------
```

```
print("In Test case 2")
B = True
E = False
A = True

p = P_B * (1 - P_E) * alarm[(B, E)] * P_Sophia
print("Probability = ", p)

# ----------------- TEST CASE 3 -----------------
print("In Test case 3")
B = False
E = True
A = True

p = (1 - P_B) * P_E * alarm[(B, E)] * P_David * (1 - P_Sophia)
print("Probability = ", p)

# ----------------- TEST CASE 4: MISSING CPT -----------------
print("In Test case 4")
if (False, False) not in alarm:
   print("missing CPT value")
else:
   print("All CPT values are present")

# ----------------- TEST CASE 5: INVALID PROBABILITY -----------------
print("In Test case 5")
x = 1.5
if x < 0 or x > 1:
   print("Invalid probability: ", x)
else   print("Valid probability")
```

EXP:4

```
from sympy import symbols, Not, Implies, Xor
from sympy.logic.inference import satisfiable

# ----------------- PROPOSITIONAL SYMBOLS -----------------
rain = symbols("rain")
harry_visited_hagrid = symbols("harry_visited_hagrid")
harry_visited_dumbledore = symbols("harry_visited_dumbledore")

# ----------------- INFERENCE FUNCTION -----------------
def run_test(test_name, kb):
  print("\n" + "=" * 50)
  print(test_name)
  print("=" * 50)

  result = satisfiable(kb)
  if result is False:
    print("Contradiction (Not satisfiable)")
  else:
    print("Satisfiable model: ", result)
    if rain in result:
      if result[rain]:
        print("It rained today")
      else:
        print("There was no rain today.")
    else:
      print("Rain cannot be determined.")

# ----------------- TEST CASE 1 -----------------
kb1 = (
```

```
    Implies(Not(rain), harry_visited_hagrid)
    & Xor(harry_visited_hagrid, harry_visited_dumbledore)
    & harry_visited_dumbledore
)
run_test("Test Case 1", kb1)

# ----------------- TEST CASE 2 -----------------
kb2 = (
    Implies(Not(rain), harry_visited_hagrid)
    & Xor(harry_visited_hagrid, harry_visited_dumbledore)
    & harry_visited_dumbledore
    & rain
)
run_test("Test Case 2", kb2)

# ----------------- TEST CASE 3 -----------------
kb3 = (
    Implies(Not(rain), harry_visited_hagrid)
    & Xor(harry_visited_hagrid, harry_visited_dumbledore)
    & harry_visited_hagrid
    & rain
)
run_test("Test Case 3", kb3)

# ----------------- TEST CASE 4 -----------------
kb4 = (
    Implies(Not(rain), harry_visited_hagrid)
    & Xor(harry_visited_hagrid, harry_visited_dumbledore)
    & harry_visited_hagrid
    & harry_visited_dumbledore
```

```
)
run_test("Test Case 4", kb4)


# ----------------- TEST CASE 5 -----------------
kb5 = harry_visited_hagrid
run_test("Test Case 5", kb5)
```

EXP:2

```
def a_star(graph, h, start, goal):
    open_list = [start]
    closed = set()
    g = {start: 0}
    parent = {start: start}

    while open_list:
        # Find node in open_list with lowest f(n) = g(n) + h(n)
        n = open_list[0]
        for node in open_list:
            if g[node] + h[node] < g[n] + h[n]:
                n = node

        # Goal check and path reconstruction
        if n == goal:
            path = []
            while parent[n] != n:
                path.append(n)
                n = parent[n]
            path.append(n)
            path.reverse()
            print("path :", path)
            print("cost :", g[goal])
            return

        # Explore neighbors
        for m, cost in graph[n]:
            new_cost = g[n] + cost
            if m not in open_list and m not in closed:
```

```
            open_list.append(m)
            parent[m] = n
            g[m] = new_cost
        elif new_cost < g.get(m, float('inf')):
            g[m] = new_cost
            parent[m] = n
            if m in closed:
                closed.remove(m)
                open_list.append(m)

    open_list.remove(n)
    closed.add(n)

  print("path not found")



# ----------------- TEST CASE 1: ADMISSIBLE HEURISTIC -----------------
print("Test Case 1: admissible heuristic")
graph1 = {
  'A': [('B', 1), ('C', 3)],
  'B': [('D', 3), ('E', 1)],
  'C': [('F', 5)],
  'D': [('G', 2)],
  'E': [('G', 4)],
  'F': [('G', 1)],
  'G': []
}
h1 = {'A': 6, 'B': 4, 'C': 4, 'D': 2, 'E': 3, 'F': 1, 'G': 0}
a_star(graph1, h1, 'A', 'G')
```

```
# ---------- TEST CASE 2: MULTIPLE POSSIBLE PATHS ----------
print("\nIn Test Case 2: multiple possible paths")
graph2 = {
  'S': [('A', 2), ('B', 4)],
  'A': [('G', 5)],
  'B': [('G', 2)],
  'G': []
}
h2 = {'S': 3, 'A': 2, 'B': 1, 'G': 0}
a_star(graph2, h2, 'S', 'G')

# ---------- TEST CASE 3: DIRECT START-GOAL CONNECTION ----------
print("\nIn Test Case 3: Direct-start Goal connection")
graph3 = {
  'A': [('G', 2), ('B', 4)],
  'B': [('G', 3)],
  'G': []
}
h3 = {'A': 2, 'B': 3, 'G': 0}
a_star(graph3, h3, 'A', 'G')

# ---------- TEST CASE 4: GOAL NOT REACHABLE ----------
print("\nIn Test Case 4: Goal not reachable")
graph4 = {
  'A': [('B', 2)],
  'B': [('C', 3)],
  'C': [],
  'G': []
}
h4 = {'A': 4, 'B': 3, 'C': 2, 'G': 0}
```

```
a_star(graph4, h4, 'A', 'G')


# ----------------- TEST CASE 5: MISLEADING HEURISTIC -----------------
print("\nIn Test Case 5: misleading heuristic")
graph5 = {
   'S': [('A', 1), ('B', 2)],
   'A': [('G', 10)],
   'B': [('G', 2)],
   'G': []
}
h5 = {'S': 1, 'A': 0, 'B': 100, 'G': 0}
a_star(graph5, h5, 'S', 'G')
```